

CircuitBreak

A Cooperative Networked Electronics Escape Room

Team: 3 members	Deadline: 14 November 2026
-----------------	----------------------------

The idea in simple words

Three players work together to repair a virtual electronics system and escape before time runs out. Each player sees different information or controls. They must talk to each other and combine their knowledge to solve three short puzzles.

Each player uses a separate client application. A central server connects everyone and keeps one shared version of the game. When one player changes a switch, the other players see the relevant results.

Why cooperation is essential

Player role	What this player can see or do
Circuit Inspector	Sees the circuit diagram, component values and instructions. Explains what the team needs to achieve.
Diagnostic Analyst	Sees measurements such as voltage, frequency and logic levels. Reports whether a change helped.
Control Operator	Changes switches, resistor settings and other controls. Uses information from the other two players.

An example round

The Inspector knows that the circuit needs a 5 V output. The Analyst sees that it is currently 3 V. The Operator changes a resistor setting, and the server calculates the new voltage. The Analyst reports the reading. The team repeats this until the target is reached and the next puzzle opens.

Why it fits the assignment

The project brief asks for real-time communication and data exchange and lists networked games as an example. The module includes socket programming (LO-5) and concurrent client-server applications using threads (LO-6). CircuitBreak can demonstrate both through connected players and shared puzzle updates.

Confirmed by the supplied documents: three members, a 14 November 2026 deadline and a project worth 15 marks. The exact programming language and permitted networking libraries are not specified.

What we will build

A small software-only game with a clear networking core.

Three proposed puzzles

Puzzle	How the team solves it
1. Restore power	Use a voltage-divider diagram, a live voltage reading and selectable resistor values to reach a safe target voltage.
2. Tune the signal	Use a simple filter diagram, a signal-strength or frequency reading and selectable resistance/capacitance values to find the required setting.
3. Unlock the logic gate	Use a logic diagram, visible output readings and input switches to discover the combination that opens the final lock.

Use simple equations and predefined circuit rules. A full circuit simulator, realistic oscilloscope and precisely timed button sequences are outside the first version.

How the network works

Three player applications	Two-way network messages	One shared game server
---------------------------	--------------------------	------------------------

- 1. Join:** Players connect to the server and receive their roles.
- 2. Act:** A client sends an action, such as changing a switch.
- 3. Update:** The server checks the action, updates the puzzle and sends each player only the information their role should receive.
- 4. Recover:** If a player disconnects, the game pauses. When they reconnect, the server restores their role and sends its current state.

Keep the first version manageable

Build one game room, one server and three clients on the same local network. Include role assignment, three puzzles, a shared timer, a win screen and basic disconnect/reconnect handling. A two-player mode can combine two roles later.

Plan for direct TCP sockets and threads, subject to lecturer confirmation. Define clear message boundaries so partially received messages can be reassembled. Protect shared game data when several client threads access it.

First decision before implementation

Ask the lecturer: **“Must we use direct socket programming, and is a particular language or library required?”** Then choose a simple interface that the team can build. Hardware buttons and LEDs remain an optional extension after the software works.

Plan to the deadline

Proposed schedule: 18 September - 14 November 2026

Dates	Deliverable and completion checkpoint
18-24 Sep	Confirm lecturer expectations and team topic. Agree on roles, puzzle rules and message format.
25 Sep - 1 Oct	Working connection prototype: three clients join one server and exchange messages.
2-8 Oct	One complete puzzle: all three roles cooperate and see consistent updates.
9-22 Oct	Add puzzles 2 and 3, puzzle transitions, timer and win screen.
23-29 Oct	Add disconnect detection and reconnect recovery. Test simultaneous actions and invalid messages.
30 Oct - 5 Nov	Run the full game across separate computers. Fix bugs and prepare setup instructions and demo notes.
6-13 Nov	Rehearse the demonstration, finish the report and keep time for fixes. Submit by 14 November.

Divide the work among three members

Member 1 - Networking: server connections, messages, threads and reconnect handling.

Member 2 - Puzzles: circuit rules, game state, timer and success conditions.

Member 3 - Client: the three role views, controls and player feedback.

Everyone: integration, testing, documentation and demonstration.

How we will know it works

Three players must complete all puzzles using their separate roles. Test that one action updates the correct views, simultaneous actions keep a consistent state, invalid messages do not crash the server, and a disconnected player can rejoin without losing progress. Save test results and a short demo recording.

Keep the plan on track

Meet twice a week and demonstrate working progress weekly. Agree on the message format before building parts separately. If the first puzzle is late, simplify graphics and circuit detail. Keep networking and all three short puzzles as the priority; add no hardware until the core game passes testing.

Resources and handover: Use available computers, a shared local network and a shared Git repository. Prepare source code, setup steps, a short explanation of the protocol and test evidence; confirm the lecturer's required submission format. Dates are targets and depend on team availability.

Basis: Supplied EC5205 module information sheet and Network Programming Project_25thBatch.pdf. Roles, puzzles, architecture and milestones are proposals, not additional lecturer requirements.